

Khulna University

Department of Computer Science and Engineering

FOIL and Beam Search Algorithm Implementation on Family Relations Dataset

Machine Learning Assignment Report

Submitted To	Professor Dr. G M Atiqur Rahaman
Course	Machine Learning
Topic	Learning First-Order Logic / Rule Learning
Department	Computer Science and Engineering
University	Khulna University
Student Name	Sumon Chandra Barman
Student ID	220212
Submission	May 2026

Abstract

This report presents the implementation of simplified FOIL (First-Order Inductive Learner) and Beam Search algorithms applied to a family relations dataset. The background predicates are `father(X,Y)` and `mother(X,Y)`. The target predicate to be learned is `grandfather(X,Y)`. Python is used to enumerate positive and negative examples, compute FOIL Gain, execute Beam Search, and generate structured output files. The algorithms successfully induce the following two rules:

```
grandfather(X,Y) :- father(X,Z), father(Z,Y).  
grandfather(X,Y) :- father(X,Z), mother(Z,Y).
```

1. Introduction

Rule learning is a supervised machine learning paradigm in which the output consists of human-readable logical rules rather than opaque model parameters. In first-order logic (FOL), these rules can express relational patterns using variables and predicates. For instance, `grandfather(X,Y)` asserts that X is the paternal grandfather of Y.

The family relations problem is a classic benchmark for Inductive Logic Programming (ILP) because it maps naturally to logical facts. Parent-child relationships are encoded as `father(X,Y)` and `mother(X,Y)` facts, while complex relations such as `grandfather(X,Y)` can be derived from these primitives using learned rules. Correctly learning the grandfather predicate requires reasoning across two levels of the family tree, making it a meaningful inductive challenge.

2. Objectives

- Implement a simplified version of the FOIL rule-learning algorithm.
- Implement a General-to-Specific Beam Search algorithm with configurable beam width.
- Apply both algorithms to the family relations dataset.
- Learn logical grandfather rules from father and mother background facts.
- Analyse and compare the performance characteristics of FOIL and Beam Search.

3. Dataset Description

The dataset contains 24 persons from two families. Background knowledge is represented as `father(X,Y)` and `mother(X,Y)` facts. The target concept to be learned is `grandfather(X,Y)`. Negative examples are generated via the Closed-World Assumption (CWA): any ordered pair not listed as a positive example is treated as negative.

Persons (24 total)

Christopher, Penelope, Arthur, Victoria, Andrew, Christine,
James, Jennifer, Colin, Charlotte, Margaret, Charles,
Roberto, Maria, Emilio, Lucia, Pierro, Francesca,
Marco, Angela, Alfonso, Sophia, Gina, Tomaso

Background Predicate 1 — father(X,Y) [12 facts]

father(Christopher, Arthur).	father(Christopher, Victoria).
father(Andrew, James).	father(Andrew, Jennifer).
father(James, Colin).	father(James, Charlotte).
father(Roberto, Emilio).	father(Roberto, Lucia).
father(Pierro, Marco).	father(Pierro, Angela).
father(Marco, Alfonso).	father(Marco, Sophia).

Background Predicate 2 — mother(X,Y) [12 facts]

mother(Penelope, Arthur).	mother(Penelope, Victoria).
mother(Christine, James).	mother(Christine, Jennifer).
mother(Victoria, Colin).	mother(Victoria, Charlotte).
mother(Maria, Emilio).	mother(Maria, Lucia).
mother(Francesca, Marco).	mother(Francesca, Angela).
mother(Lucia, Alfonso).	mother(Lucia, Sophia).

Target Predicate — grandfather(X,Y) [8 positive examples]

Positive examples are automatically computed: X is grandfather of Y if X is father of some Z, and Z is a parent of Y.

grandfather(Christopher, Colin).	grandfather(Christopher, Charlotte).
grandfather(Andrew, Colin).	grandfather(Andrew, Charlotte).
grandfather(Roberto, Alfonso).	grandfather(Roberto, Sophia).
grandfather(Pierro, Alfonso).	grandfather(Pierro, Sophia).

The dataset has 8 positive examples and 568 negative examples (24 x 24 - 8), generated by CWA.

4. FOIL Algorithm

FOIL (First-Order Inductive Learner), introduced by Quinlan (1990), is a sequential covering algorithm that constructs first-order Horn clauses from labelled examples. Starting with the most general clause — an empty rule body — FOIL iteratively specialises the current clause by appending the literal that maximises FOIL Gain until no negative examples remain covered.

Initial Rule

```
grandfather(X,Y) :- true.
```

Procedure

```
Initialise: clause body = {} (most general rule)
While negatives are still covered by the current clause:
  Generate candidate literals
  Compute FOIL Gain for each candidate
  Select the literal L with maximum gain
  Append L to the clause body
Remove positives covered by the completed clause
Repeat from beginning if positives remain
```

Candidate Literals

```

father(X,Z)    % X is father of some intermediate Z
father(Z,Y)    % Z is father of Y
mother(Z,Y)    % Z is mother of Y
father(X,Y)    % X is directly father of Y
mother(X,Z)    % X is mother of some Z
mother(X,Y)    % X is directly mother of Y

```

5. FOIL Gain

FOIL Gain quantifies the information gain obtained by adding a new literal to the current partial clause. A candidate literal with high gain retains many positive bindings while eliminating negative ones.

Formula

$$\text{Gain}(L, R) = t \times [\log_2(p_1 / (p_1 + n_1)) - \log_2(p_0 / (p_0 + n_0))]$$

Symbol	Description
p0	Positive bindings before adding literal L
n0	Negative bindings before adding literal L
p1	Positive bindings after adding literal L
n1	Negative bindings after adding literal L
t	Number of positive tuples still covered by the clause

6. Beam Search Algorithm

Beam Search is a heuristic best-first search strategy that generalises greedy search by maintaining a beam of the top-k candidate hypotheses at each refinement step. Unlike FOIL, which commits irrevocably to a single best literal, Beam Search explores a broader portion of the hypothesis space, reducing the risk of being trapped in a locally optimal but globally suboptimal clause.

Key Parameters

- Beam width $k = 3$: number of candidate rules maintained at each refinement level.
- Max depth = 4: maximum number of literals added to a rule.
- Scoring metric: composite of accuracy (precision) and coverage (recall).
- Termination: when no new literal improves the top beam candidate or all positives are covered.

Scoring Formulas

```

Accuracy = p / (p + n)
Coverage = p / total_positives

```

7. Implementation Details

Project Folder Structure

```
family_ilp/
|
|-- family_dataset.py
|
|-- family_loader.py
|-- family_foil.py
|-- family_beam_search.py
|-- main_family.py
`-- output/
    |-- family_foil_output.txt
    `-- family_beam_output.txt
```

File Descriptions

File	Purpose
family_dataset.py	Stores all persons, father/mother facts, auto-computes grandfather examples
family_loader.py	Loads data, generates negative examples, defines candidate literals
family_foil.py	Implements simplified FOIL rule learner
family_beam_search.py	Implements General-to-Specific Beam Search
main_family.py	Orchestrates the full pipeline and writes output files
output/family_foil_output.txt	FOIL algorithm results
output/family_beam_output.txt	Beam Search results

8. Python Implementation Explanation

The dataset is stored in data/family_dataset.py. The father and mother facts are manually listed. The GRANDFATHER examples are automatically computed by a helper function that finds all (X, Y) pairs where X is father of some Z and Z is a parent of Y.

Auto-computation of Grandfather Examples

```
def _compute_grandfather(father_facts, mother_facts):
    parent_facts = set(father_facts) | set(mother_facts)
    grandchildren_of = {}
    for x, z in father_facts:
        grandchildren_of.setdefault(x, set())
        for z2, y in parent_facts:
            if z2 == z:
                grandchildren_of[x].add(y)
    return [(x, y) for x, ys in grandchildren_of.items() for y in sorted(ys)]
```

FOIL Learning — Rule 1

FOIL starts with `grandfather(X,Y) :- true.` covering all 8 positives and 568 negatives. The best literal at Step 1 is `father(X,Z)` with FOIL Gain = 16.0000. After adding it: `p1=8, n1=136`. At Step 2, best literal is `father(Z,Y)` with gain = 16.6797. After adding: `p1=4, n1=0` — no negatives remain, rule is pure.

```

Learned Rule 1: grandfather(X,Y) :- father(X,Z), father(Z,Y).
Covers: (Andrew,Colin), (Andrew,Charlotte), (Pierro,Alfonso), (Pierro,Sophia)

```

FOIL Learning — Rule 2

Remaining 4 positives. Again best at Step 1 is `father(X,Z)` with gain = 8.1224. At Step 2, `father(Z,Y)` yields 0 positives (those are already covered). Best is `mother(Z,Y)` with gain = 20.5171. After adding: `p1=4, n1=0` — rule is pure.

```

Learned Rule 2: grandfather(X,Y) :- father(X,Z), mother(Z,Y).
Covers: (Christopher,Colin), (Christopher,Charlotte), (Roberto,Alfonso),
(Roberto,Sophia)

```

All 8 positives are now covered. FOIL finishes with 2 rules.

9. FOIL Output (Actual)

```

FOIL Learning - Target: grandfather(X,Y)
Positive examples : 8
Negative examples : 568

```

```

=== Learning Rule 1 ===
Start: grandfather(X,Y) :- true.

```

```

Coverage: p0=8, n0=568
Evaluating candidate literals:
  father(X,Z)      p1= 8, n1=136, gain=16.0000
  father(Z,Y)      p1= 8, n1=280, gain= 8.0000
  mother(Z,Y)      p1= 8, n1=280, gain= 8.0000
  father(X,Y)      p1= 0, n1= 12, gain= -inf
  mother(X,Z)      p1= 0, n1=144, gain= -inf
  mother(X,Y)      p1= 0, n1= 12, gain= -inf

```

```

>> Selected: father(X,Z) (gain=16.0000)
Updated rule : grandfather(X,Y) :- father(X,Z).
Coverage now : p1=8, n1=136

```

```

Coverage: p0=8, n0=136
  father(Z,Y)      p1= 4, n1= 0, gain=16.6797 << BEST
  mother(Z,Y)      p1= 4, n1= 0, gain=16.6797

```

```

>> Selected: father(Z,Y) (gain=16.6797)
Updated rule : grandfather(X,Y) :- father(X,Z), father(Z,Y).
Coverage now : p1=4, n1=0
No negatives covered - rule is pure. Done.

```

```

Learned Rule 1: grandfather(X,Y) :- father(X,Z), father(Z,Y).

```

```
=== Learning Rule 2 ===
>> Selected: father(X,Z) (gain=8.1224)
>> Selected: mother(Z,Y) (gain=20.5171)
Learned Rule 2: grandfather(X,Y) :- father(X,Z), mother(Z,Y).

Final Learned Rules:
grandfather(X,Y) :- father(X,Z), father(Z,Y).
grandfather(X,Y) :- father(X,Z), mother(Z,Y).
FOIL Finished.
```

10. Beam Search Output (Actual)

Beam Search - Target: grandfather(X,Y)
Beam width=3, Max depth=4

```
=== Searching Rule 1 ===
Depth 1: 6 candidates
1. grandfather(X,Y) :- father(X,Z). acc=0.056, cov=1.000, p=8, n=136
2. grandfather(X,Y) :- father(Z,Y). acc=0.028, cov=1.000, p=8, n=280
3. grandfather(X,Y) :- mother(Z,Y). acc=0.028, cov=1.000, p=8, n=280

Depth 2: 15 candidates
1. grandfather(X,Y) :- father(X,Z), father(Z,Y). acc=1.000, cov=0.500, p=4, n=0
2. grandfather(X,Y) :- father(X,Z), mother(Z,Y). acc=1.000, cov=0.500, p=4, n=0

Selected Rule 1: grandfather(X,Y) :- father(X,Z), father(Z,Y).

=== Searching Rule 2 ===
Depth 2: 15 candidates
1. grandfather(X,Y) :- father(X,Z), mother(Z,Y). acc=1.000, cov=1.000, p=4, n=0

Selected Rule 2: grandfather(X,Y) :- father(X,Z), mother(Z,Y).

Final Rules:
grandfather(X,Y) :- father(X,Z), father(Z,Y).
grandfather(X,Y) :- father(X,Z), mother(Z,Y).
Beam Search Finished.
```

11. Result Analysis

Both algorithms learned the same two rules:

```
grandfather(X,Y) :- father(X,Z), father(Z,Y).
grandfather(X,Y) :- father(X,Z), mother(Z,Y).
```

Rule 1 — Paternal grandfather via son: X is grandfather of Y if X is the father of some Z, and Z is also the father of Y. Example: grandfather(Andrew, Colin) because father(Andrew, James) and father(James, Colin).

Rule 2 — Paternal grandfather via daughter: X is grandfather of Y if X is the father of some Z, and Z is the mother of Y. Example: grandfather(Christopher, Colin) because father(Christopher, Victoria) and mother(Victoria, Colin).

Together these two rules cover all 8 positive examples with zero false positives (100% accuracy, 100% coverage).

12. Comparison: FOIL vs. Beam Search

Criteria	FOIL	Beam Search
Search Type	General-to-Specific	General-to-Specific
Selection Method	FOIL Gain (greedy)	Best k candidates
Memory Usage	Lower	Higher (beam width)
Search Strategy	Single best literal	Multiple candidates (k=3)
Result Quality	Good (may miss optimal)	Often better
Interpretability	High	High
Final Rules	2 rules learned	2 rules learned
Accuracy	100%	100%

On this dataset both methods arrive at identical rules. The advantage of Beam Search becomes significant on larger, noisier datasets where greedy literal selection in FOIL can lead to suboptimal clauses.

13. Why Prolog Is Not Used Here

The original graph assignment used Prolog to manually write rules and verify queries. In this assignment, the goal is to automatically learn those Prolog-style rules from data using Python. So:

- Python acts as the learner — it discovers the rules using FOIL and Beam Search.
- Prolog would be the output — the learned rules can directly be written in Prolog.
- Writing rules manually in Prolog would not be machine learning.

The equivalent Prolog representation of the learned rules would be:

```
% Background knowledge
father(christopher, arthur).    father(christopher, victoria).
% ... (all 12 father facts)
mother(penelope, arthur).      mother(penelope, victoria).
% ... (all 12 mother facts)

% Learned rules
grandfather(X,Y) :- father(X,Z), father(Z,Y).
grandfather(X,Y) :- father(X,Z), mother(Z,Y).

% Sample queries:
?- grandfather(christopher, colin).    % true
```



```
?- grandfather(andrew, charlotte).      % true
?- grandfather(penelope, colin).        % false
```

14. Limitations

- The FOIL implementation is simplified and does not handle negated literals or function symbols.
- Only grandfather is learned as the target predicate. Other relations like uncle, aunt, grandmother could also be learned with different candidate literals.
- The CWA generates 568 negative examples vs only 8 positives, creating class imbalance.
- FOIL is greedy — it may miss a globally better rule if a locally weaker literal leads to a better overall rule.
- Beam width is fixed at 3; sensitivity analysis across different widths was not conducted.

15. Conclusion

This assignment demonstrated the practical application of two foundational rule-learning algorithms — FOIL and Beam Search — to a family relations dataset. Both algorithms successfully induced a two-clause hypothesis that correctly captures the grandfather relation:

```
grandfather(X,Y) :- father(X,Z), father(Z,Y).
grandfather(X,Y) :- father(X,Z), mother(Z,Y).
```

Rule 1 handles the case where the grandfather's son is the father of the grandchild. Rule 2 handles the case where the grandfather's daughter is the mother of the grandchild. Together they achieve 100% accuracy and 100% coverage over all 8 positive examples with zero false positives. This exercise highlights how ILP can automatically discover meaningful relational rules from structured data without manual rule writing — a capability beyond the reach of propositional learners.

References

- [1] Quinlan, J. R. (1990). Learning Logical Definitions from Relations. *Machine Learning*, 5(3), 239-266.
- [2] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
- [3] Course Lecture Slides: Learning First-Order Logic / Rule Learning, *Machine Learning*, Khulna University.
- [4] Lavrac, N., & Dzeroski, S. (1994). *Inductive Logic Programming: Techniques and Applications*. Ellis Horwood.